

GROOT: Effective Design of Biological Sequences with Limited Experimental Data

Thanh V. T. Tran*
FPT Software AI Center
Hanoi, Vietnam
ThanhTVT1@fpt.com

Viet Anh Nguyen
FPT Software AI Center
Hanoi, Vietnam
AnhNV117@fpt.com

Nhat Khang Ngo*
FPT Software AI Center
Ho Chi Minh, Vietnam
KhangNN4@fpt.com

Truong Son Hy†
University of Alabama at Birmingham
Birmingham, AL 35294, United States
thy@uab.edu

Abstract

Latent space optimization (LSO) is a powerful method for designing discrete, high-dimensional biological sequences that maximize expensive black-box functions, such as wet lab experiments. This is accomplished by learning a latent space from available data and using a surrogate model f_Φ to guide optimization algorithms toward optimal outputs. However, existing methods struggle when labeled data is limited, as training f_Φ with few labeled data points can lead to subpar outputs, offering no advantage over the training data itself. We address this challenge by introducing **GROOT**, a **GR**aph-based Latent **SmOO**thing for Biological Sequence Optimization. In particular, GROOT generates pseudo-labels for neighbors sampled around the training latent embeddings. These pseudo-labels are then refined and smoothed by Label Propagation. Additionally, we theoretically and empirically justify our approach, demonstrate GROOT's ability to extrapolate to regions beyond the training set while maintaining reliability within an upper bound of their expected distances from the training regions. We evaluate GROOT on various biological sequence design tasks, including protein optimization (GFP and AAV) and three tasks with exact oracles from Design-Bench. The results demonstrate that GROOT equalizes and surpasses existing methods without requiring access to black-box oracles or vast amounts of labeled data, highlighting its practicality and effectiveness. We release our code at <https://github.com/Fsoft-AIC/GROOT>.

CCS Concepts

• **Computing methodologies** → **Neural networks**; • **Applied computing** → **Molecular evolution**; **Bioinformatics**.

Keywords

Protein Optimization; Latent Space Optimization; Landscape Smoothing; Label Propagation.

*Equal contribution.

†Corresponding author.



This work is licensed under a Creative Commons Attribution International 4.0 License.

KDD '25, Toronto, ON, Canada

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1245-6/25/08

<https://doi.org/10.1145/3690624.3709291>

ACM Reference Format:

Thanh V. T. Tran, Nhat Khang Ngo, Viet Anh Nguyen, and Truong Son Hy. 2025. GROOT: Effective Design of Biological Sequences with Limited Experimental Data. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '25)*, August 3–7, 2025, Toronto, ON, Canada. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3690624.3709291>

1 Introduction

Proteins are crucial biomolecules that play diverse and essential roles in every living organism. Enhancing protein functions or cellular fitness is vital for industrial, research, and therapeutic applications [17, 40]. One powerful approach to achieve this is *directed evolution*, which involves iteratively performing random mutations and screening for proteins with desired phenotypes [40]. However, the vast space of possible proteins makes exhaustive searches impractical in nature, the laboratory, or computationally. Consequently, recent machine learning (ML) methods have been developed to improve the sample efficiency of this evolutionary search [19, 25, 33]. When experimental data is available, these approaches use a surrogate model f_Φ , trained to guide optimization algorithms toward optimal outputs.

While these methods have achieved state-of-the-art results on various benchmarks, most neglect the scenario of extremely limited labeled data and fail to utilize the abundant unlabeled data. This is a significant issue because exploring protein function and characteristics through iterative mutation in wet-lab is costly [14]. As later shown in Section 4.2, previous works perform poorly in this context, proving that training f_Φ with few labeled data points can lead to subpar designs, offering no advantage over the training data itself. One potential solution for dealing with noisy and limited data is to *regularize* the fitness landscape model, smoothing the sequence representation and facilitating the use of gradient-based optimization algorithms. Although some works have addressed this problem (Section 2), none can optimize effectively in the case of labeled scarcity.

1.1 Contribution

Our work builds on the observation by Trabucco et al. [37] that surrogate models (f_Φ) trained on limited labeled data are vulnerable to noisy labels, often sampling false negatives or getting stuck in local minima during offline optimization. To address this, we

introduce **GROOT**, a framework that improves surrogate model’s predictive ability by generating synthetic sample representations from existing data. Using encoded features, we construct a graph with fitness values as node attributes and apply Label Propagation [42] to generate labels for interpolated nodes. The smoothed data is then fitted with a neural network for optimization. Figure 1 illustrates our framework. Evaluated on Green Fluorescent Proteins (GFP) [34] and Adeno-Associated Virus (AAV) [8], GROOT consistently outperforms prior baselines across all difficulty levels. Unlike other methods, it remains robust even with fewer than 100 labeled training sequences. Our key contributions are summarized below:

- We introduce GROOT, a novel framework that uses graph-based smoothing to train a smoothed fitness model, which is then used in the optimization process.
- We theoretically show that our smoothing technique can expand into extrapolation regions while keeping a reasonable distance from the training data. This helps to reduce errors occurring when the surrogate model makes predictions for unseen points too far from the training set.
- We empirically show that GROOT achieves state-of-the-art results across all difficulties in two protein design benchmarks, AAV and GFP. Notably, in extreme cases with very limited labeled data, our approach performs stably, achieving 6 fold fitness improvement in GFP and 1.3 times higher in AAV compared to the training set.
- We further evaluate GROOT on diverse tasks of different domains (e.g., robotics, DNA) within the Design-Bench. Our experimental results are competitive with state-of-the-art approaches, highlighting the method’s domain-agnostic capabilities.

2 Related work

LSO for Sequence Design. Directed evolution is a traditional paradigm in protein sequence design that has achieved notable successes [2]. Within this framework, various machine learning algorithms have been proposed to improve the sample efficiency of evolutionary searches [13, 31, 33, 39]. However, most of these methods optimize protein sequences directly in the sequence space, dealing with discrete, high-dimensional decision variables. Alternatively, Gómez-Bombarelli et al. [16] and Castro et al. [9] employed a VAE model and applied gradient ascent to optimize the latent representation, which is then decoded into biological sequences. Similarly, Lee et al. [24] used an off-policy reinforcement learning method to facilitate updates in the representation space. Another notable approach [36] involves training a denoising autoencoder with a discriminative multi-task Gaussian process head, enabling gradient-based optimization of multi-objective acquisition functions in the latent space of the autoencoder. Recently, latent diffusion has been introduced for designing novel proteins, leveraging the capabilities of a protein language model [12].

Protein Fitness Regularization. The NK model was an early attempt to model protein fitness smoothness statistically [20]. Gómez-Bombarelli et al. [16] mapped sequences to a regularized latent fitness landscape with an auxiliary network predicting fitness. Castro et al. [9] improved this with negative sampling and interpolation-based regularization. Frey et al. [15] proposed learning a smoothed

energy function, enabling sequence sampling via Markov Chain Monte Carlo (MCMC). Closest to our work, GGS [22] utilize discrete regularization using graph-based smoothing techniques. However, GGS assumes similar sequences have similar fitness, which may cause suboptimal results as fitness can vary with a single mutation [5]. Our work addresses this by encoding sequences into latent space and building a graph based on latent vector similarity. Furthermore, by operating in latent space, our framework extends beyond biological domains to applications like robotics (Appendix D).

3 Method

We begin this section by formally defining the problem. Next, we detail the construction of the protein latent space and present our proposed latent graph-based smoothing framework, GROOT. Figure 1 provides a visual overview of our method.

3.1 Problem Formulation

We address the challenge of designing proteins to find high-fitness sequences s within the sequence space $\mathcal{A}^L$, where $\mathcal{A}$ represents the amino acid vocabulary (i.e. $|\mathcal{A}| = 20$ since both animal and plant proteins are made up of about 20 common amino acids) and L is the desired sequence length. Our goal is to design sequences that maximize a black-box protein fitness function $\mathcal{O} : \mathcal{A}^L \mapsto \mathbb{R}$, which can only be evaluated through wet-lab experiments:

$$s^* = \operatorname{argmax}_{s \in \mathcal{A}^L} \mathcal{O}(s). \quad (1)$$

For in-silico evaluation, given a static dataset of all known sequences and fitness measurements $\mathcal{D}^* = \{(s, y) | s \in \mathcal{A}^L, y \in \mathbb{R}\}$, an oracle $\mathcal{O}_\psi$ parameterized by ψ is trained to minimize the prediction error on $\mathcal{D}^*$. Afterward, $\mathcal{O}_\psi$ is used as an approximator for the black-box function $\mathcal{O}$ to evaluate computational methods that are developed on a training subset $\mathcal{D}$ of $\mathcal{D}^*$. In other words, given $\mathcal{D}$, our task is to generate sequences $\hat{s}$ that optimize the fitness approximated by $\mathcal{O}_\psi$. To do this, we train a surrogate model f on $\mathcal{D}$ and use it to guide the search for optimal candidates, which are later evaluated by $\mathcal{O}_\psi$. This setup is similar to work done by Jain et al. [19], Kirjner et al. [22].

3.2 Constructing Latent Space of Protein Sequences

Unsupervised learning has achieved remarkable success in domains like natural language processing [7] and computer vision [21]. This method efficiently learns data representations by identifying underlying patterns without requiring labeled data. The label-free nature of unsupervised learning aligns well with protein design challenges, where experimental fitness evaluations are costly, while unlabeled sequences are abundant. In this work, considering a dataset of unlabeled sequences of a protein family $s \in \mathcal{A}^L$ denoted as $\mathcal{D}_u = \{s_i\}_{i=1}^{N_u}$, we train a VAE comprising an encoder $\phi : \mathcal{A}^L \mapsto \mathbb{R}^{d_h}$ and a decoder $\theta : \mathbb{R}^d \mapsto \mathcal{A}^L$. Each sequence s_i is encoded into a low-dimensional vector $h_i \in \mathbb{R}^{d_h}$. Subsequently, the mean $\mu_i \in \mathbb{R}^d$ and log-variance $\log \sigma_i \in \mathbb{R}^d$ of the variational posterior approximation are computed from h_i using two feed-forward networks, $\mu_\phi : \mathbb{R}^{d_h} \mapsto \mathbb{R}^d$ and $\sigma_\phi : \mathbb{R}^{d_h} \mapsto \mathbb{R}^d$. A latent vector $x_i \in \mathbb{R}^d$ is then sampled from the Gaussian distribution $\mathcal{N}(\mu_i, \sigma_i^2)$,

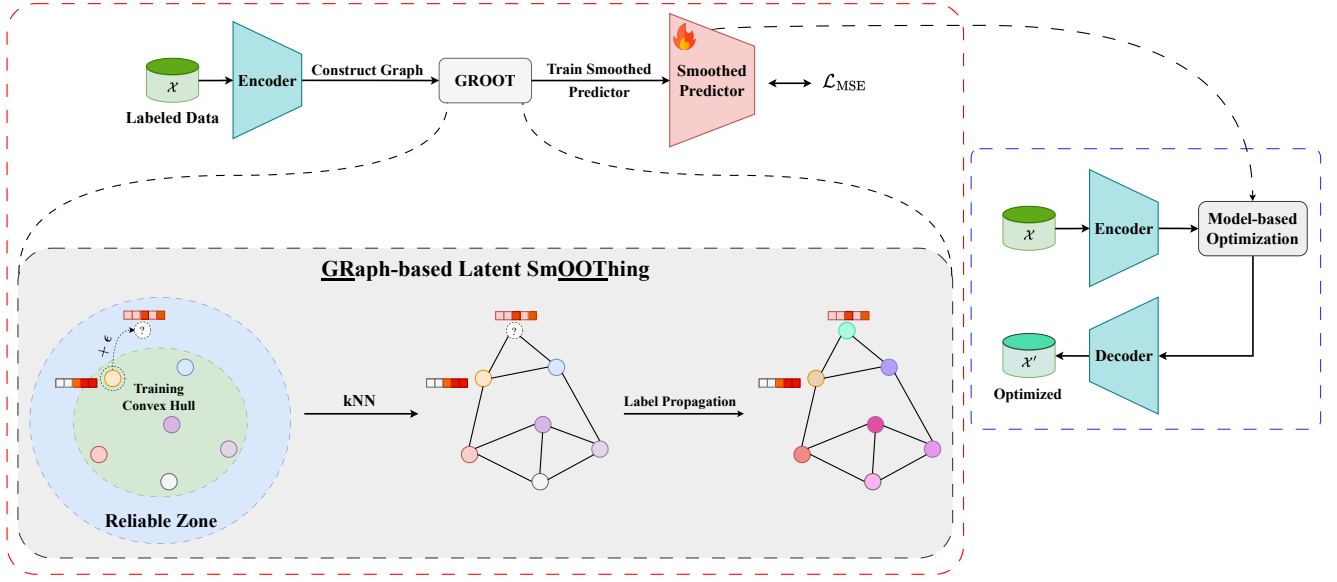


Figure 1: Overall framework of GROOT. After encoding sequences into the latent space, we generate new samples by adding Gaussian noise to existing vectors. These synthetic data lie outside the training set’s convex hull but within a reliable zone, as their distances from the hull are below a certain upper bound. We construct a kNN graph and run label propagation to smooth and refine node labels. These nodes and their fitness values are then used to train the surrogate model, which is subsequently employed for optimization.

and the decoder θ maps x_i back to the reconstructed protein sequence $\hat{s}_i \in \mathcal{A}^L$. The training objective involves the cross-entropy loss $C(\hat{s}_i, s_i)$ between the ground truth sequence s_i and the generated $\hat{s}_i$, as well as the Kullback-Leibler (KL) divergence between $\mathcal{N}(\mu_i, \sigma_i^2)$ and $\mathcal{N}(0, I_d)$:

$$\mathcal{L}_{vae} = \frac{1}{N_u} \sum_{i=1}^{N_u} C(\hat{s}_i, s_i) + \frac{\eta}{N_u} \sum_{i=1}^{N_u} D_{KL}(\mathcal{N}(\mu_i, \sigma_i^2) \parallel \mathcal{N}(0, I_d)). \quad (2)$$

Here, η is the hyperparameter to control the disentanglement property of the VAE’s latent space. A critical distinction must be made between the latent space of protein sequences and the protein fitness landscape. While training a VAE-based model, we do *not* access protein fitness scores, but we learn their continuous representations. Consequently, this VAE-based model is trained on the entirety of available protein family sequences. Subsequently, labeled sequences are encoded by the pretrained VAE into d -dimensional latent points, enabling the supervised training of a surrogate model, $f_\phi : \mathbb{R}^d \mapsto \mathbb{R}$, to predict fitness values and approximate the fitness landscape.

3.2.1 VAE’s Architecture. We go into detail regarding the architecture of the VAE used in our study.

Encoder. incorporates a pre-trained ESM-2 [26] followed by a latent encoder to compute the latent representation z . In our study, we leverage the powerful representation of the pre-trained 30-layer ESM-2 by making it the encoder of our model. Given an input sequence $s = \langle s_0, s_1, \dots, s_L \rangle$, where $s_i \in \mathcal{A}$, ESM-2 computes representations for each token s_i in s , resulting in a token-level hidden representation $H = \langle h_0, h_1, \dots, h_L \rangle, h_i \in \mathbb{R}^{d_h}$. We calculate the global representation $h \in \mathbb{R}^{d_h}$ of s via a weighted sum of its

tokens:

$$h = \sum_{i=1}^L \frac{\omega^T \exp(h_i)}{\sum_{i=1}^L \omega^T \exp(h_i)} h_i. \quad (3)$$

here, ω is a learnable global attention vector. Then, two multi-layer perceptrons (MLPs) are used to compute $\mu = \text{MLP}_1(h)$ and $\log \sigma = \text{MLP}_2(h)$, where the latent dimension is d . Finally, a latent representation $x \in \mathbb{R}^d$ is sampled from $\mathcal{N}(\mu, \sigma^2)$, which is further proceeded to the decoder to reconstruct the sequence $\hat{s}$.

Decoder. To decode latent points into sequences, we employ a deep convolutional network as proposed by Castro et al. [9], consisting of four one-dimensional convolutional layers. ReLU activations and batch normalization layers are applied between the convolutional layers, except for the final layer.

3.3 Latent Graph-based Smoothing

Graph Construction. We need to build a graph $G = (\mathcal{V}, \mathcal{E})$ from the sequences in $\mathcal{D}$ to perform label propagation. Algorithm 2 demonstrates our algorithm to construct a kNN graph from the latent vectors x . The graph’s nodes are created by sampling N sequences $s \in \mathcal{D}$, and the edges are constructed by k -nearest neighbors to the latent embeddings of $x = \phi(s)$, where ϕ is the pre-trained encoder. We introduce new nodes to G by interpolating the learned latent x with random noise $\epsilon \in \mathcal{N}(0, I_d)$. We argue that Levenshtein distance is an inadequate metric for computing protein sequence similarity as done in [22] because marginal amino acid variations can result in substantial fitness disparities [6, 29]. Meanwhile, high-dimensional latent embeddings are effective at capturing implicit patterns within protein sequences. Consequently,

Euclidean distance can serve as a suitable metric to quantify sequence similarity for constructing kNN graphs, as proteins with comparable properties (indicated by small fitness score differences) tend to cluster closely in the latent space. Algorithm 2 presents our graph construction pipeline.

Algorithm 1 kNN: k-Nearest Neighbors

Require: Current node x , All nodes $\mathcal{V}$

- 1: $D(x) \leftarrow \bigcup_{x' \in \mathcal{V} \setminus \{x\}} \|x' - x\|$ ▷ Euclidean distance.
 - 2: $\mathcal{X}' \leftarrow \text{TopK}(D(x), \mathcal{V})$ ▷ Retrieve K closest nodes to x .
 - 3: $\mathcal{E}(x) \leftarrow \bigcup_{x' \in \mathcal{X}'} (x, x')$ ▷ Construct neighborhood around x .
 - 4: **return** $\mathcal{E}(x)$
-

Algorithm 2 CreateGraph: Latent Graph Construction

Require: training embeddings $\mathcal{V}$, #of graph nodes N , constant β

- 1: **while** $|\mathcal{V}| < N$ **do**
 - 2: $\bar{x} \sim \mathcal{U}(\mathcal{V})$
 - 3: $\epsilon \sim \mathcal{N}(0, I_d)$ ▷ Sample Gaussian noise ϵ .
 - 4: $z \leftarrow \beta * \bar{x} + (1 - \beta) * \epsilon$ ▷ Interpolate between $\bar{x}$ and ϵ .
 - 5: $\mathcal{V} \leftarrow \mathcal{V} \cup \{z\}$
 - 6: **end while**
 - 7: $\mathcal{E} \leftarrow \bigcup_{x \in \mathcal{V}} \text{kNN}(x, \mathcal{V})$ ▷ Construct edges (Algorithm 1)
 - 8: **return** $G = (\mathcal{V}, \mathcal{E})$
-

Generating Pseudo-label. We consider a constructed graph G . For each node $v \in \mathcal{V}$, we assign a fitness label y , creating a set of fitness labels Y . This set Y can be further decomposed into $Y = \{Y_n, Y_u\}$, representing the known and unknown labels, respectively. Known labels Y_n are obtained from the training dataset, while unknown labels Y_u are initialized by 0 and assigned to randomly generated nodes, as shown in line 3 of Algorithm 3. We then run the following label propagation in m times.

$$Y' = \alpha D^{-1/2} A D^{-1/2} Y + (1 - \alpha) Y, \quad (4)$$

here, $\alpha \in [0, 1]$ is a weighted coefficient and A is a weighted adjacency matrix, defined as:

$$A_{ij} = \begin{cases} \frac{\gamma}{d(x_i, x_j)} & \text{if } i \neq j, \\ 0 & \text{if } i = j, \end{cases} \quad (5)$$

and $d(\cdot, \cdot)$ is the Euclidean distance, γ is a controllable factor. In summary, Algorithm 3 details our smoothing strategy by label propagation.

Algorithm 3 Smooth: Latent Graph-based Smoothing

Require: graph $G = (\mathcal{V}, \mathcal{E})$, label propagation layers m

- 1: $A \leftarrow \text{WeightedAdjacencyMatrix}(\mathcal{E})$ ▷ Equation (5)
 - 2: $D \leftarrow \text{DegreeMatrix}(\mathcal{V})$ ▷ Compute degree matrix.
 - 3: $Y \leftarrow [0 \text{ if is_synthetic else } y_i]_{i=1}^N$
 - 4: **for** $i = 0, \dots, m - 1$ **do**
 - 5: $Y \leftarrow \text{LabelPropagation}(A, D, Y)$ ▷ Equation (4)
 - 6: **end for**
 - 7: **return** Y
-

3.4 Theoretical Justification

This section delves into the theoretical foundations of our proposed method. We begin by discussing the concepts of interpolation and extrapolation within high-dimensional latent embedding spaces. These concepts ensure the correctness of generating additional pseudo-samples within the latent space of a VAE-based model. Subsequently, leveraging the definition of convex hulls, we establish an upper bound for the expected distance between the generated nodes and the collection of training latent embeddings. This bound informs the intuition behind our kNN graph construction, detailed in Algorithm 2. Consequently, employing label propagation within this bounded distance emerges as a rational strategy for assigning pseudo-labels to the newly generated nodes.

We first list the definitions of a convex hull, interpolation, and extrapolation as follows:

Definition 3.1. Let $\mathbb{X} = \{x_1, \dots, x_N\}$ be a set of N points in $\mathbb{R}^d$, a convex hull of $\mathbb{X}$ is defined as:

$$\text{Conv}(\mathbb{X}) \triangleq \left\{ \sum_{i=1}^N \lambda_i x_i \mid \lambda_i \geq 0 \text{ and } \sum_{i=1}^N \lambda_i = 1 \right\}.$$

Definition 3.2 (Balestriero et al. [3]). Interpolation occurs for a sample x whenever this sample belongs to $\text{Conv}(\mathbb{X})$, if not, extrapolation occurs.

Assumptions. Given a set of N training protein sequences $\mathbb{S} = \{s_1, s_2, \dots, s_N\}$, we use the encoder ϕ of the pretrained VAE to compute their latent embeddings, resulting in a set of latent vectors $\mathbb{X} = \{x_1, x_2, \dots, x_N\}$, where $x_i = \phi(s_i)$ (see Section 3.2). In this paper, we consider the scenario wherein we have limited access to experimental data, and we train a VAE-based model to represent continuous d -dimensional latent vectors of those available sequences. This, therefore, gives rise to two mild assumptions in our work.

- **Assumption 1:** N is significantly smaller than the exponential in d , where N represents the number of data samples.
- **Assumption 2:** Each latent vector x is sampled from $\mathcal{N}(0, I_d)$. This is achieved by regularizing the model with KL divergence loss as shown in the Equation (2).

We assess GROOT's extrapolation capabilities by computing the probability of its generated samples falling outside the convex hull of the training data within the latent space. Subsequently, we derive an upper bound for these reliable extrapolation regions. These findings are formalized in the following propositions.

PROPOSITION 1. Let $\mathbb{X} = \{x_i \in \mathbb{R}^d\}_{i=1}^N$ be a set of N i.i.d d -dimensional samples from $\mathcal{N}(0, I_d)$. Assume that $N \ll \exp\left(\frac{d}{2(C_\beta^2 + 2)}\right)$

and let $z = \beta * \bar{x} + (1 - \beta) * \epsilon \in \mathbb{X}$, $\beta \in (0, 1)$, $C_\beta = \frac{1+\beta}{1-\beta}$ and $\epsilon \sim \mathcal{N}(0, I_d)$, then

$$\lim_{d \rightarrow \infty} \mathbb{P}(z \notin \text{Conv}(\mathbb{X})) = 1.$$

We leave the proof of the presented proposition in Appendix A.1. Proposition 1 elucidates that in a limited data scenario (i.e. $N \ll \exp\left(\frac{d}{2(C_\beta^2 + 2)}\right)$), the probability for a synthetic latent node z lying outside the convex hull of training set $\mathbb{X}$ goes to 1 as the latent

Algorithm 4 GROOT-augmented MBO

Require: training dataset $\mathcal{D}$, pre-trained encoder ϕ , # of graph nodes N , constant β , label propagation layers m

- 1: $\mathcal{V} \leftarrow \{x_i = \phi(s_i), s_i \in \mathcal{D}\}_{i=1}^n$ $\triangleright$ Compute latent embeddings
- 2: $\mathcal{V}', \mathcal{E} \leftarrow \text{CreateGraph}(\mathcal{V}, N, \beta)$ $\triangleright$ Algorithm 2
- 3: $\hat{Y} \leftarrow \text{Smooth}((\mathcal{V}', \mathcal{E}), L)$ $\triangleright$ Algorithm 3
- 4: $\Phi \leftarrow \arg \min_{\Phi} \mathbb{E}_{(x,y) \sim (\mathcal{V}', \hat{Y})} [(y - f_{\Phi}(x))^2]$ $\triangleright$ Train model
- 5: $X^*, Y^* \leftarrow \text{MBO}(\Phi, \mathcal{V}')$ $\triangleright$ Model-based optimization
- 6: **return** X^*, Y^*

dimension d grows. Based on Definition 3.2, this theoretical result guarantees the correctness of our formula shown in line 4 of Algorithm 2 in expanding the latent space of protein sequences. However, when generated nodes are located far from the training convex hull, their embedding vectors differ significantly from those in the training set. This disparity hinders the identification of meaningful similarities and can result in unreliable fitness scores determined through label propagation. This brings us to Proposition 2.

PROPOSITION 2. Let $\mathbb{X} = \{x_i \in \mathbb{R}^d\}_{i=1}^N$ be a set of N i.i.d d -dimensional samples from $\mathcal{N}(0, I_d)$. For any $\bar{x} \in \mathbb{X}$, we define $z = \beta * \bar{x} + (1 - \beta) * \epsilon$ with $\epsilon \sim \mathcal{N}(0, I_d)$, $\beta \in (0, 1)$, the following holds:

$$\mathbb{E}[D(z, \text{Conv}(\mathbb{X}))] < 2(1 - \beta)\sqrt{d},$$

where $D(z, \text{Conv}(\mathbb{X})) \triangleq \inf_{x \in \text{Conv}(\mathbb{X})} d(z, x)$ is the distance from a point $z \in \mathbb{R}^d$ to $\text{Conv}(\mathbb{X})$, and $d(\cdot, \cdot)$ is the Euclidean distance.

The proof of this proposition can be found in Appendix A.2. Proposition 2 allows us to quantify the expected distance from a randomly generated node based on our formula to the available training set. The derived upper bound increases linearly w.r.t square root of the latent dimension d with a rate of $2(1 - \beta)$. It is worth noting that β is used to control the exploration rate of our algorithm. As $\beta \rightarrow 1$, the generated node z closely resembles a training node $\bar{x}$. Conversely, a lower β introduces more noise into the latent vectors, resulting in samples that are further from the source nodes. This upper bound ensures that synthetic nodes remain within a controllable "reliable zone", making label propagation a sensible choice, as the label values of these synthetic nodes cannot deviate too far from the overall distribution of existing nodes.

In summary, Propositions 1 and 2 offer theoretical underpinnings for our latent-based smoothing approach. By mapping discrete sequences to a continuous d -dimensional latent space, we establish a quantitative relationship between newly added and original training nodes, governed by parameters d and β . This quantitative connection enhances the interpretability of our method.

3.5 Model-based Optimization Algorithm

Latent space model-based optimization (LSO) uses surrogate models $f_{\Phi} : \mathbb{R}^d \rightarrow \mathbb{R}$ to efficiently search for optimal values that maximize costly black-box functions. The success of LSO relies on surrogate accuracy, as poorly trained models can mislead optimization. As shown in Algorithm 4, lines 1, 4, and 5 describe the standard MBO process: encoding training vectors into latent space and training a surrogate to predict fitness scores Y . Lines 2 and 3 introduce GROOT

Table 1: Comparison of GROOT and the baselines on 3 Design-Bench tasks. Bold results indicate the best value, and underlined results indicate the second-best value. The standard deviation of 3 runs with different random seeds is indicated in parentheses.

	Method	D’Kitty	Ant	TF Bind 8	Mean Score $\uparrow$
Median	MINs	<u>0.86</u> (0.01)	0.49 (0.15)	0.42 (0.02)	0.59
	BONET	0.85 (0.01)	0.60 (0.12)	<u>0.44</u> (0.00)	0.63
	BDI	0.59 (0.02)	0.40 (0.02)	0.54 (0.03)	0.51
	ExPT	0.90 (0.01)	0.71 (0.02)	0.47 (0.01)	0.69
	GROOT	0.82 (0.01)	<u>0.62</u> (0.01)	<u>0.52</u> (0.01)	<u>0.65</u>
Max	MINs	0.93 (0.01)	0.89 (0.01)	0.81 (0.03)	0.88
	BONET	0.91 (0.01)	0.84 (0.04)	0.69 (0.15)	0.81
	BDI	0.92 (0.01)	0.81 (0.09)	0.91 (0.07)	0.88
	ExPT	<u>0.97</u> (0.01)	0.97 (0.00)	<u>0.93</u> (0.04)	<u>0.96</u>
	GROOT	0.98 (0.01)	0.97 (0.00)	0.98 (0.02)	0.98
Mean	MINs	0.62 (0.03)	0.01 (0.01)	0.42 (0.03)	0.35
	BONET	<u>0.84</u> (0.02)	0.58 (0.02)	0.45 (0.01)	0.62
	BDI	0.57 (0.03)	0.39 (0.01)	<u>0.54</u> (0.03)	0.50
	ExPT	0.87 (0.02)	0.64 (0.03)	0.48 (0.01)	0.66
	GROOT	0.78 (0.03)	<u>0.60</u> (0.01)	0.55 (0.02)	<u>0.64</u>

’s additional steps before surrogate training. GROOT is domain-agnostic, allowing any encoding method or surrogate model, making it a versatile tool for optimization across diverse domains.

4 Experiments

We demonstrate the benefits of our proposed GROOT on two protein tasks with varying levels of label scarcity. Furthermore, since obtaining the ground-truth fitness of generated sequences is costly, we further evaluate our method on Design-Bench [37]. These benchmarks have exact oracles, providing a better validation of our method’s performance on real-world datasets.

Design-Bench. We provide detailed experimental setup in Appendix D. Table 1 shows the performance of various methods across three Design-Bench tasks and their mean scores. While most methods perform well, GROOT achieves the highest average score in maximum performance and is slightly behind the SOTA baseline in median and mean benchmarks. Since these tasks are evaluated by an exact oracle and our performance is competitive with other SOTA methods, this confirms that our approach is effective on real-world data, enhancing the reliability of our work.

4.1 Experiment Setup

Datasets and Oracles. Following Kirjner et al. [22], we evaluate our method on two proteins, GFP [34] and AAV [8]. The length L is 237 for GFP and 28 for the functional segment of AAV. The full GFP dataset $\mathcal{D}^*$ comprises 54,025 mutant sequences with associated log-fluorescence intensity, while the AAV dataset contains 44,156 sequences linked to their ability to package a DNA payload. The fitness values in both datasets are min-max normalized for

Table 2: Statistic of benchmarks.

Task	Difficulty	Fitness Range (%)	Mutational Gap	Best Fitness	$ \mathcal{D} $
AAV	Harder1	< 30th	13	0.33	1157
	Harder2	< 20th	13	0.29	920
	Harder3	< 10th	13	0.24	476
GFP	Harder1	< 30th	8	0.10	1129
	Harder2	< 20th	8	0.01	792
	Harder3	< 10th	8	0.01	397

evaluation but remain unchanged during training and inference. To demonstrate the advantages of our smoothing method, we use the *harder1*, *harder2*, and *harder3* level benchmarks proposed by Kirjner et al. [22] to sample training datasets $\mathcal{D}$ for each task, simulating scenarios with scarce labeled data¹. Table 2 provides statistics for each level benchmark. All methods, including baselines, start optimization using the entire training set $\mathcal{D}$ and are evaluated on the best 128 generated sequences, with approximated fitness predicted by oracles provided by Kirjner et al. [22]. It is important to note that oracles do not participate in the optimization process and are only used as *in-silico* evaluators to validate final proposed designs of each method.

Baselines. We evaluate our method against several representative baselines using the open-source toolkit FLEXS [35]: AdaLead [35], CbAS [4], and Bayesian Optimization (BO) [41]. In addition to these algorithms, we also benchmark against some most recent methods: GFN-AL [19], ReLSO [9], PEX [33], and GGS [22], utilizing and modifying the code provided by their respective authors. Furthermore, we assess ReLSO enhanced with our smoothing strategy for better optimization, termed S-ReLSO.

Evaluation Metrics. We use three metrics proposed by Jain et al. [19]: fitness, diversity, and novelty. Let the optimized sequences $\mathcal{G}^* = \{g_1^*, \dots, g_K^*\}$. Fitness is defined as the median of the evaluated fitness of $K = 128$ proposed designs. Diversity is the median of the distances between every pair of sequences in $\mathcal{G}^*$. Novelty is defined as the median of the distances between every generated sequences to the training set.

Implementation Details. For each task, we use the full dataset $\mathcal{D}^*$ to train the VAE, whose architecture has been described in Section 3.2. We utilize the pretrained checkpoint `esm2_t30_150M_UR50D`² of the ESM-2 model, fine-tuning only the last layer while freezing the rest. The latent representation space dimension is set to $d = 320$. For the latent graph-based smoothing, we set the number of nodes to $N = 20,000$, label propagation’s coefficient to $\alpha = 0.2$, controller factor to $\gamma = 1.0$, the number of propagation layers to $N_{\text{layers}} = 1$, and the number of neighbors to $k = 8$. It is important to note that these hyperparameters are not finely tuned. The hyperparameter tuning process is described in the Appendix C. After the smoothing process, the features and refined labels are inputted into a surrogate model, specifically a shallow 2-layer multi-layers perceptron (MLP)

with a dropout rate of 0.2 in the first layer, to train a smoothed surrogate model. Despite its simplicity, we consider MLP a suitable choice due to its proven effectiveness in previous studies [18]. For model-based optimization, we select two gradient-based algorithms to exploit the smoothness of the surrogate model: Gradient Ascent (GA) with a learning rate of 0.005 for 400 iterations and Limited-memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS) [27] for a maximum of 6 iterations. While various optimization algorithms are available, we choose these gradient-based methods based on the premise that they can effectively leverage the smoothness of the surrogate model. It is crucial to note that *our primary focus is not on evaluating multiple optimization methods, but rather on demonstrating the effectiveness of the smoothing strategy*. We empirically evaluate these two algorithms and report the best results.

4.2 Numerical Results

We report the mean and standard deviation of the evaluation metrics over five runs with different random seeds in Table 3. Firstly, compared to the best fitness of the training data (Table 2), our method successfully optimizes all tasks. For AAV tasks, our method achieves SOTA results across all difficulty levels. Notably, although our method only slightly exceeds AdaLead in the *harder2* task, GROOT generates sequences whose novelty falls within the mutation gap range, indicating appropriate extrapolation and a higher likelihood of producing synthesizable proteins. Additionally, applying our smoothing strategy to ReLSO improves the model’s performance by 60% in the *harder1*, 64.7% in the *harder2*, and 22.7% in the *harder3* tasks. For GFP tasks, GROOT outperforms all baseline methods across all difficulties, especially in the *harder3* difficulty, where GGS failed to optimize properly. It is crucial to mention that ReLSO failed to generate diverse sequences in GFP tasks, as indicated by zero diversity across five different runs. Overall, we observe that GROOT achieves the highest fitness while maintaining respectable diversity and novelty.

4.3 Analysis

Impact of smoothing on extrapolation and general performance. For each benchmark, we assess the impact of smoothing on extrapolation capabilities by measuring the Mean Absolute Error (MAE) of the surrogate model on the benchmark’s training and holdout datasets relative to the experimental ground-truth. Table 4 shows the benefits of smoothing on extrapolation to held out ground-truth experimental data. Our results demonstrate that smoothing reduces MAE for both the training and holdout sets. This reduction occurs because our smoothing strategy acts as a data augmentation technique in the latent space, enhancing the robustness of the supervised model. We also observe that the MAE for both sets increases gradually as the task becomes more difficult. We hypothesize that this is due to the smaller training set size in more difficult tasks, requiring a greater number of new samples to construct a graph. While the labels of these new samples are not exact, they help to smooth the latent landscape, resulting in a smoother but not perfectly accurate surrogate model, which in turn increases the MAE.

Additionally, Table 5 demonstrates how a smoothed model dramatically outperforms its unsmoothed counterpart across all tasks.

¹Results of other difficulties are presented in the Appendix B

²https://huggingface.co/facebook/esm2_t30_150M_UR50D

Table 3: AAV and GFP optimization results for GROOT and baseline methods. The standard deviation of 5 runs with different random seeds is indicated in parentheses.

Method	AAV <i>harder1</i> task			AAV <i>harder2</i> task			AAV <i>harder3</i> task		
	Fitness ↑	Diversity	Novelty	Fitness ↑	Diversity	Novelty	Fitness ↑	Diversity	Novelty
AdaLead	0.38 (0.0)	5.5 (0.5)	7.0 (0.7)	0.43 (0.0)	4.2 (0.7)	7.8 (0.8)	0.37 (0.0)	6.22 (0.9)	8.0 (1.2)
CbAS	0.02 (0.0)	22.9 (0.1)	18.5 (0.5)	0.01 (0.0)	23.2 (0.1)	19.3 (0.4)	0.01 (0.0)	23.2 (0.1)	19.3 (0.4)
BO	0.00 (0.0)	20.4 (0.3)	21.8 (0.4)	0.01 (0.0)	20.4 (0.0)	22.0 (0.0)	0.01 (0.0)	20.6 (0.3)	22.0 (0.0)
GFN-AL	0.00 (0.0)	15.4 (6.2)	21.6 (0.5)	0.00 (0.0)	8.1 (3.5)	21.6 (1.0)	0.00 (0.0)	7.6 (0.8)	22.6 (1.4)
PEX	0.23 (0.0)	6.4 (0.5)	3.8 (0.7)	0.30 (0.0)	7.8 (0.4)	5.0 (0.0)	0.26 (0.0)	7.3 (0.7)	4.4 (0.5)
GGs	0.30 (0.0)	13.6 (0.2)	14.5 (0.3)	0.27 (0.0)	16.0 (0.0)	19.4 (0.0)	0.38 (0.0)	7.0 (0.1)	9.6 (0.1)
ReLSO	0.15 (0.0)	20.9 (0.0)	13.0 (0.0)	0.17 (0.0)	20.3 (0.0)	13.0 (0.0)	0.22 (0.0)	17.8 (0.0)	11.0 (0.0)
S-ReLSO	0.24 (0.0)	11.5 (0.0)	13.0 (0.0)	0.28 (0.0)	16.4 (0.0)	6.5 (0.0)	0.27 (0.0)	17.7 (0.0)	11.0 (0.0)
GROOT (GA)	0.37 (0.0)	13.6 (0.9)	10.0 (0.7)	0.36 (0.0)	13.7 (1.1)	10.1 (0.9)	0.34 (0.1)	14.0 (2.2)	10.0 (1.4)
GROOT	0.46 (0.1)	9.8 (1.6)	12.2 (0.5)	0.45 (0.0)	9.9 (0.8)	13.0 (0.0)	0.42 (0.1)	11.0 (2.0)	13.0 (0.0)

Method	GFP <i>harder1</i> task			GFP <i>harder2</i> task			GFP <i>harder3</i> task		
	Fitness ↑	Diversity	Novelty	Fitness ↑	Diversity	Novelty	Fitness ↑	Diversity	Novelty
AdaLead	0.39 (0.0)	8.4 (3.2)	9.0 (1.2)	0.4 (0.0)	7.3 (2.8)	9.8 (0.4)	0.42 (0.0)	6.4 (2.3)	9.0 (1.2)
CbAS	-0.08 (0.0)	172.2 (35.7)	201.5 (1.5)	-0.09 (0.0)	158.4 (34.8)	202.0 (0.7)	-0.08 (0.0)	186.4 (33.4)	201.5 (0.9)
GFN-AL	0.21 (0.1)	74.3 (55.3)	219.2 (3.3)	0.14 (0.2)	27.0 (9.5)	223.5 (2.4)	0.21 (0.0)	37.5 (21.7)	219.8 (4.3)
PEX	0.13 (0.0)	12.6 (1.2)	7.1 (1.1)	0.17 (0.0)	12.6 (1.2)	7.1 (1.1)	0.19 (0.0)	12.2 (1.1)	7.8 (1.7)
GGs	0.67 (0.0)	4.7 (0.2)	9.1 (0.1)	0.60 (0.0)	5.4 (0.2)	9.8 (0.1)	0.00 (0.0)	15.7 (0.4)	19.0 (2.2)
ReLSO	0.94 (0.0) [†]	0.0 (0.0)	8.0 (0.0)	0.94 (0.0) [†]	0.0 (0.0)	8.0 (0.0)	0.94 (0.0) [†]	0.0 (0.0)	8.0 (0.0)
S-ReLSO	0.94 (0.0) [†]	0.0 (0.0)	8.0 (0.0)	0.94 (0.0) [†]	0.0 (0.0)	8.0 (0.0)	0.94 (0.0) [†]	0.0 (0.0)	8.0 (0.0)
GROOT	0.88 (0.0)	3.0 (0.2)	7.0 (0.0)	0.87 (0.0)	3.0 (0.1)	7.5 (0.5)	0.62 (0.2)	7.6 (1.5)	8.6 (1.5)

[†] indicates that the generated population has collapsed (i.e., producing only a single sequence).

For AAV, the results show up to a threefold improvement compared to the unsmoothed surrogate model. For GFP, without smoothing, the optimization algorithm struggles to find a maximum point, resulting in suboptimal performance. With our smoothing technique, the optimization algorithm can optimize properly. Our findings indicate that our smoothing strategy enables gradient-based optimization methods to better navigate the search space and identify superior maxima. However, we also observe that applying smoothing decreases diversity. This can be explained by the optimization process converging, causing the population to concentrate in certain high-fitness regions. In contrast, without proper optimization, the population diverges and spreads throughout the search space.

Empirical analysis of our propositions. In this section, we directly validate our proposed Propositions 1 and 2 through experiments. Firstly, we train multiple VAEs with the same architecture described in Section 3.2, modifying only the latent dimension size d . For each VAE version, we construct a graph and apply the smoothing technique as described in Section 3.3 to generate new nodes. Secondly, to determine whether the generated nodes lie within the convex hull, we check if each point can be expressed as a convex combination of the training points. This is formulated as a linear programming problem and can be easily solved using existing libraries³. After identifying the number of nodes lying outside the convex hull, we calculate the Euclidean distance $D(z, \mathbb{X})$ between the outside nodes and the set of training nodes.

³<https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html>

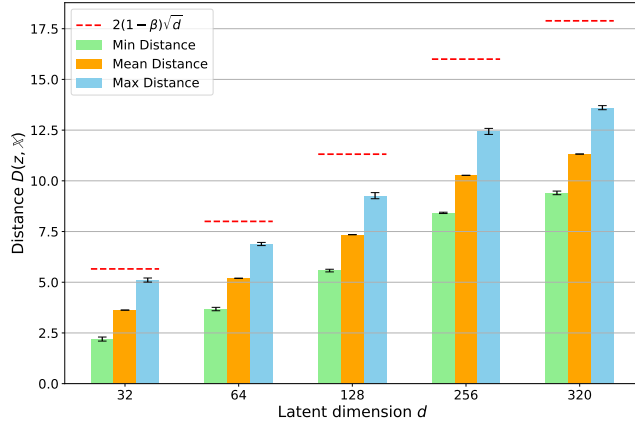
Table 4: Smoothing improves extrapolation on every tasks. The mean of 5 different runs is reported.

Task	Difficulty	Smoothed	Train MAE ↓	Holdout MAE ↓
AAV	Harder1	No	4.94	8.93
		Yes	1.02	5.78
	Harder2	No	4.67	8.91
		Yes	1.10	6.11
	Harder3	No	4.13	8.87
		Yes	1.44	7.30
GFP	Harder1	No	1.39	2.81
		Yes	0.22	1.71
	Harder2	No	1.34	2.81
		Yes	0.31	1.85
	Harder3	No	1.33	2.80
		Yes	0.51	2.09

We validate our propositions on the GFP harder3 task, setting $\beta = 0.5$ consistently across all runs to maintain a constant upper bound. Our experiments reveal that, regardless of dimensional size, 100% of the synthetic nodes lie outside the convex hull. Figure 2 demonstrates distance calculations between the outside nodes and

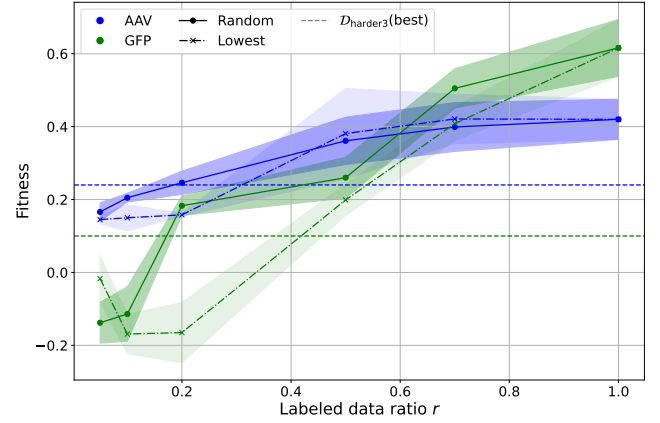
Table 5: Smoothing improves performance on every tasks. The mean of 5 different runs is reported.

Task	Difficulty	Smoothed	Fitness $\uparrow$	Diversity	Novelty
AAV	Harder1	No	0.12	20.0	10.0
		Yes	0.46	9.8	12.2
	Harder2	No	0.11	20.0	9.6
		Yes	0.45	9.9	13.0
	Harder3	No	0.12	20.1	10.0
		Yes	0.42	11.0	13.0
GFP	Harder1	No	-0.12	71.0	42.2
		Yes	0.88	3.0	7.0
	Harder2	No	-0.18	69.5	41.1
		Yes	0.87	3.0	7.5
	Harder3	No	-0.17	64.0	37.0
		Yes	0.62	7.6	8.6

**Figure 2: Distance from generated nodes outside the convex hull to the set of training nodes. The mean and standard deviation over 5 different runs are reported. We set $\beta = 0.5$ to maintain a constant upper bound on the distance.**

the set of training nodes. It is evident that although the distance increases with the latent dimension, none of the generated nodes in any VAE versions exceed the proven upper bound. This demonstrates that our smoothing technique can expand into extrapolation regions while remaining within a suitable range, avoiding distances too far from the training set that could introduce errors in surrogate model predictions.

Explore the limits of smoothing strategy. In this section, we investigate the smoothing capabilities of our method by testing its limits. We use the GFP *harder3* task and the AAV *harder3* task as examined cases, further subsampling the dataset with ratios $r \in \{0.05, 0.1, 0.2, 0.5, 0.7, 1.0\}$ in two different ways, namely: *random* and *lowest*. In the *random* setting, data points are randomly subsampled with the given ratio r , while in the *lowest* setting, we use the fraction r of the dataset with the lowest fitness scores.

**Figure 3: The performance of GROOT on AAV and GFP *harder3* tasks when we vary the labeled data ratio r . The mean and standard deviation over 5 different runs are reported.**

Using the same hyperparameters, Figure 3 shows the median performance of GROOT on AAV and GFP w.r.t the ratio r in both settings. In the *random* setting, our method outperforms the best data point in the entire *harder3* dataset using only 20% of the labeled data, which amounts to fewer than 100 samples per set. In the *lowest* setting, the method requires 50% of the labeled data, approximately 200 samples per set, to achieve similar performance. This demonstrates that our method is effective even under extreme conditions with limited labeled data.

Efficiency of GROOT. In this part, we analyze the time complexity of our smoothing algorithm, excluding the extracting embeddings from training data and training surrogate model with smoothed data. We first divide our method into two phases and then analyze them individually:

- **Create graph** (Algorithm 2): Initially, sampling x from $\mathcal{V}$ and generating Gaussian noise ϵ are $O(1)$ and $O(d)$ operations, respectively. The interpolation to create z and adding z to $\mathcal{V}$ are $O(d)$ and $O(1)$. These steps occur in each iteration of the while loop, which runs $N - |\mathcal{V}|$ times, resulting in a total complexity of $O((N - |\mathcal{V}|) \cdot d)$. The most significant part of the complexity arises from the edge construction (Algorithm 1). Calculating Euclidean distances between x and all other nodes in $\mathcal{V}$ involves d operations for each of the $|\mathcal{V}| - 1$ distances, leading to $O(d \cdot |\mathcal{V}|)$. Retrieving the K closest nodes is $O(|\mathcal{V}| \log K)$, and constructing the neighborhood around x is $O(K)$. Since these steps must be performed for each node in $\mathcal{V}$, the overall time complexity for edge construction is $O(|\mathcal{V}| \cdot (d \cdot |\mathcal{V}|)) + O(|\mathcal{V}| \cdot (|\mathcal{V}| \log K)) + O(|\mathcal{V}| \cdot K)$, simplifying to $O(d \cdot |\mathcal{V}|^2)$ since $\log K \ll d$. Given that $|\mathcal{V}|$ eventually reaches N , the overall time complexity is $T_1 = O(d \cdot N^2)$.
- **Smooth** (Algorithm 3): The algorithm begins with constructing the weighted adjacency matrix A as defined in Equation (5). This construction involves calculating the Euclidean distance between each pair of nodes. Since this computation has been done in the previous step, the time complexity of this step is $O(N^2)$. As shown in Equation 4, a label propagation layer calculates the

inverse square root of the degree matrix, $D^{-1/2}$, and multiplies it by the adjacency matrix, A . Efficiently, the inverse of the diagonal matrix D can be computed in linear time, $O(N)$. According to Raghuvaran et al. [32], label propagation can be interpreted as label exchange across edges, resulting in a time complexity of $O(|E|)$, significantly less than the naive $O(N^3)$ matrix multiplication. Thus, the computation time involves $O(N^2)$ for constructing A and D , $O(N)$ for taking the inversion, and $O(m \cdot |E|)$ for running label propagation in m layers. Moreover, since kNN graphs exhibit sparse structures with numerous sub-communities, the adjacency matrix A is also sparse, and thus $O(m \cdot |E|) \ll O(m \cdot N^2)$ in our work. Therefore, the overall time complexity is $T_2 = 2O(N^2) + O(N) + O(m \cdot |E|) \approx O(N^2)$.

In summary, GROOT has a time complexity of $T = T_1 + T_2 = O(d \cdot N^2) + O(N^2)$. Since $O(N^2)$ is dominated by $O(d \cdot N^2)$, T simplifies to $O(d \cdot N^2)$.

5 Conclusion

This paper presents a novel method, GROOT, to address the scarcity of labeled data in practical protein sequence design. GROOT embeds protein sequences into a continuous latent space. In this space, we generate additional synthetic latent points by interpolating existing data points and Gaussian noise and estimating their fitness scores using label propagation. We provide theoretical underpinnings based on convex hull and extrapolation to support GROOT's efficacy. Experimental results confirm the method's effectiveness in protein design under extreme labeled data-limited conditions.

Limitations. Our proposed method, GROOT, has some limitations with hyperparameters, which are sensitive to landscape characteristics and require optimization for new landscapes. The hyperparameter tuning process, detailed in Appendix C, shows that while sensitivity varies across datasets, optimal hyperparameters remain stable within the same landscape's difficulty levels, reducing the search burden. Additionally, we theoretically link GROOT's hyperparameters to sampled nodes, ensuring framework controllability. A promising direction is to improve the determination of β by analyzing random mutations in protein sequence families.

References

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-generation Hyperparameter Optimization Framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (Anchorage, AK, USA) (KDD '19)*. Association for Computing Machinery, New York, NY, USA, 2623–2631. <https://doi.org/10.1145/3292500.3330701>
- [2] Frances H. Arnold. 2018. Directed Evolution: Bringing New Chemistry to Life. *Angewandte Chemie International Edition* 57, 16 (2018), 4143–4148. <https://doi.org/10.1002/anie.201708408> arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1002/anie.201708408>
- [3] Randall Balestriero, Jerome Pesenti, and Yann LeCun. 2021. Learning in high dimension always amounts to extrapolation. *arXiv preprint arXiv:2110.09485* (2021).
- [4] David Brookes, Hahnbeom Park, and Jennifer Listgarten. 2019. Conditioning by adaptive sampling for robust design. In *Proceedings of the 36th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 97)*, Kamalika Chaudhuri and Ruslan Salakhutdinov (Eds.). PMLR, 773–782. <https://proceedings.mlr.press/v97/brookes19a.html>
- [5] David H. Brookes, Amirali Aghazadeh, and Jennifer Listgarten. 2022. On the sparsity of fitness functions and implications for learning. *Proceedings of the National Academy of Sciences* 119, 1 (2022), e2109649118. <https://doi.org/10.1073/pnas.2109649118> arXiv:<https://www.pnas.org/doi/pdf/10.1073/pnas.2109649118>
- [6] David H. Brookes, Amirali Aghazadeh, and Jennifer Listgarten. 2022. On the sparsity of fitness functions and implications for learning. *Proceedings of the National Academy of Sciences* 119, 1 (2022), e2109649118.
- [7] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 1877–1901. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf
- [8] Drew H. Bryant, Ali Bashir, Sam Sinai, Nina K. Jain, Pierce J. Ogden, Patrick F. Riley, George M. Church, Lucy J. Colwell, and Eric D. Kelsic. 2021. Deep diversification of an AAV capsid protein by machine learning. *Nature Biotechnology* 39, 6 (01 Jun 2021), 691–696. <https://doi.org/10.1038/s41587-020-00793-4>
- [9] Egbert Castro, Abhinav Godavarti, Julian Rubinien, Kevin Givechian, Dhananjay Bhaskar, and Smita Krishnaswamy. 2022. Transformer-based protein generation with regularized latent space optimization. *Nature Machine Intelligence* 4, 10 (01 Oct 2022), 840–851. <https://doi.org/10.1038/s42256-022-00532-1>
- [10] Venkat Chandrasekaran, Benjamin Recht, Pablo A Parrilo, and Alan S Willsky. 2012. The convex geometry of linear inverse problems. *Foundations of Computational mathematics* 12, 6 (2012), 805–849.
- [11] Can Chen, Yingxue Zhang, Jie Fu, Xue (Steve) Liu, and Mark Coates. 2022. Bidirectional Learning for Offline Infinite-width Model-based Optimization. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 29454–29467. https://proceedings.neurips.cc/paper_files/paper/2022/file/bd391cf5bdc4b63674d6da3edc1bde0d-Paper-Conference.pdf
- [12] Tianlai Chen, Pranay Vure, Rishab Pulugurta, and Pranam Chatterjee. 2023. AMP-Diffusion: Integrating Latent Diffusion with Protein Language Models for Antimicrobial Peptide Generation. In *NeurIPS 2023 Generative AI and Biology (GenBio) Workshop*. <https://openreview.net/forum?id=145TM9VQhx>
- [13] Patrick Emami, Aidan Perreault, Jeffrey Law, David Biagioni, and Peter St. John. 2023. Plug and play directed evolution of proteins with gradient-based discrete MCMC. *Machine Learning: Science and Technology* 4, 2 (April 2023), 025014. <https://doi.org/10.1088/2632-2153/accad>
- [14] Douglas M. Fowler and Stanley Fields. 2014. Deep mutational scanning: a new style of protein science. *Nature Methods* 11, 8 (01 Aug 2014), 801–807. <https://doi.org/10.1038/nmeth.3027>
- [15] Nathan C. Frey, Dan Berenberg, Karina Zadorozhny, Joseph Kleinhenz, Julien Lafrance-Vanasse, Isidro Hotzel, Yan Wu, Stephen Ra, Richard Bonneau, Kyunghyun Cho, Andreas Loukas, Vladimir Gligorijevic, and Saeed Saremi. 2024. Protein Discovery with Discrete Walk-Jump Sampling. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=zMPHKOmQNb>
- [16] Rafael Gómez-Bombarelli, Jennifer N. Wei, David Duvenaud, José Miguel Hernández-Lobato, Benjamin Sánchez-Lengeling, Dennis Sheberla, Jorge Aguilera-Iparraguirre, Timothy D. Hirzel, Ryan P. Adams, and Alan Aspuru-Guzik. 2018. Automatic Chemical Design Using a Data-Driven Continuous Representation of Molecules. *ACS Central Science* 4, 2 (2018), 268–276. <https://doi.org/10.1021/acscentsci.7b00572> arXiv:<https://doi.org/10.1021/acscentsci.7b00572> PMID: 29532027
- [17] Po-Ssu Huang, Scott E. Boyken, and David Baker. 2016. The coming of age of de novo protein design. *Nature* 537, 7620 (01 Sep 2016), 320–327. <https://doi.org/10.1038/nature19946>
- [18] Qian Huang, Horace He, Abhay Singh, Ser-Nam Lim, and Austin Benson. 2021. Combining Label Propagation and Simple Models out-performs Graph Neural Networks. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=8E1-f3VhX1o>
- [19] Moksh Jain, Emmanuel Bengio, Alex Hernandez-Garcia, Jarrid Rector-Brooks, Bonaventure F. P. Dossou, Chanakya Ajit Ekbote, Jie Fu, Tianyu Zhang, Michael Kilgour, Dinghuai Zhang, Lena Simine, Payel Das, and Yoshua Bengio. 2022. Biological Sequence Design with GFlowNets. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*, Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato (Eds.). PMLR, 9786–9801. <https://proceedings.mlr.press/v162/jain22a.html>
- [20] Stuart A. Kauffman and Edward D. Weinberger. 1989. The NK model of rugged fitness landscapes and its application to maturation of the immune response. *Journal of Theoretical Biology* 141, 2 (Nov. 1989), 211–245. [https://doi.org/10.1016/s0022-5193\(89\)80019-0](https://doi.org/10.1016/s0022-5193(89)80019-0)
- [21] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. 2023. Segment Anything. *arXiv:2304.02643* (2023).

- [22] Andrew Kirjner, Jason Yim, Raman Samusevich, Shahar Bracha, Tommi S. Jaakkola, Regina Barzilay, and Ila R Fiete. 2024. Improving protein optimization with smoothed fitness landscapes. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=rxlF2Zv8x0>
- [23] Aviral Kumar and Sergey Levine. 2020. Model Inversion Networks for Model-Based Optimization. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 5126–5137. https://proceedings.neurips.cc/paper_files/paper/2020/file/373e4c5d8edfa8b74fd4b6791d0cf6dc-Paper.pdf
- [24] Minji Lee, Luiz Felipe Vecchiatti, Hyunkyung Jung, Hyunjo Ro, Meeyoung Cha, and Ho Min Kim. 2023. Protein Sequence Design in a Latent Space via Model-based Reinforcement Learning. <https://openreview.net/forum?id=OhjGzRE5N6o>
- [25] Minji Lee, Luiz Felipe Vecchiatti, Hyunkyung Jung, Hyunjo Ro, Meeyoung Cha, and Ho Min Kim. 2024. Robust Optimization in Protein Fitness Landscapes Using Reinforcement Learning in Latent Space. In *Forty-first International Conference on Machine Learning*. <https://openreview.net/forum?id=0zbxwvJqwf>
- [26] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Zmetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom Sercu, Salvatore Candido, and Alexander Rives. 2023. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science* 379, 6637 (2023), 1123–1130. <https://doi.org/10.1126/science.ade2574> arXiv:<https://www.science.org/doi/pdf/10.1126/science.ade2574>
- [27] Dong C. Liu and Jorge Nocedal. 1989. On the limited memory BFGS method for large scale optimization. *Mathematical Programming* 45, 1 (01 Aug 1989), 503–528. <https://doi.org/10.1007/BF01589116>
- [28] Satvik Mehul Mashkaria, Siddharth Krishnamoorthy, and Aditya Grover. 2023. Generative Pretraining for Black-Box Optimization. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 24173–24197. <https://proceedings.mlr.press/v202/mashkaria23a.html>
- [29] John Maynard Smith. 1970. Natural Selection and the Concept of a Protein Space. *Nature* 225, 5232 (Feb. 1970), 563–564. <https://doi.org/10.1038/225563a0>
- [30] Tung Nguyen, Sudhanshu Agrawal, and Aditya Grover. 2023. ExPT: Synthetic Pretraining for Few-Shot Experimental Design. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 45856–45869. https://proceedings.neurips.cc/paper_files/paper/2023/file/8fab4407e1fe9006b39180525c0d323c-Paper-Conference.pdf
- [31] Yuchi Qiu and Guo-Wei Wei. 2022. CLADE 2.0: Evolution-Driven Cluster Learning-Assisted Directed Evolution. *Journal of Chemical Information and Modeling* 62, 19 (Sept. 2022), 4629–4641. <https://doi.org/10.1021/acs.jcim.2c01046>
- [32] Usha Nandini Raghavan, Réka Albert, and Soundar Kumara. 2007. Near linear time algorithm to detect community structures in large-scale networks. *Phys. Rev. E* 76 (Sep 2007), 036106. Issue 3. <https://doi.org/10.1103/PhysRevE.76.036106>
- [33] Zhizhou Ren, Jiahua Li, Fan Ding, Yuan Zhou, Jianzhu Ma, and Jian Peng. 2022. Proximal Exploration for Model-guided Protein Sequence Design. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*, Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato (Eds.). PMLR, 18520–18536. <https://proceedings.mlr.press/v162/ren22a.html>
- [34] Karen S. Sarkisyan, Dmitry A. Bolotin, Margarita V. Meer, Dinara R. Usmanova, Alexander S. Mishin, George V. Sharonov, Dmitry N. Ivankov, Nina G. Bozhanova, Mikhail S. Baranov, Onuralp Soylemez, Natalya S. Bogatyreva, Peter K. Vlasov, Evgeny S. Egorov, Maria D. Logacheva, Alexey S. Kondrashov, Dmitry M. Chudakov, Ekaterina V. Putintseva, Ilgar Z. Mamedov, Dan S. Tawfik, Konstantin A. Lukyanov, and Fyodor A. Kondrashov. 2016. Local fitness landscape of the green fluorescent protein. *Nature* 533, 7603 (01 May 2016), 397–401. <https://doi.org/10.1038/nature17995>
- [35] Sam Sinai, Richard Wang, Alexander Whatley, Stewart Slocum, Elina Locane, and Eric D. Kelsic. 2020. AdaLead: A simple and robust adaptive greedy search algorithm for sequence design. *CoRR* abs/2010.02141 (2020). arXiv:2010.02141 <https://arxiv.org/abs/2010.02141>
- [36] Samuel Stanton, Wesley Maddox, Nate Gruver, Phillip Maffettone, Emily Delaney, Peyton Greenside, and Andrew Gordon Wilson. 2022. Accelerating Bayesian Optimization for Biological Sequence Design with Denoising Autoencoders. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*, Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato (Eds.). PMLR, 20459–20478. <https://proceedings.mlr.press/v162/stanton22a.html>
- [37] Brandon Trabucco, Xinyang Geng, Aviral Kumar, and Sergey Levine. 2022. Design-Bench: Benchmarks for Data-Driven Offline Model-Based Optimization. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*, Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato (Eds.). PMLR, 21658–21676. <https://proceedings.mlr.press/v162/trabucco22a.html>
- [38] Brandon Trabucco, Aviral Kumar, Xinyang Geng, and Sergey Levine. 2021. Conservative Objective Models for Effective Offline Model-Based Optimization. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 139)*, Marina Meila and Tong Zhang (Eds.). PMLR, 10358–10368. <https://proceedings.mlr.press/v139/trabucco21a.html>
- [39] Thanh V. T. Tran and Truong Son Hy. 2024. Protein Design by Directed Evolution Guided by Large Language Models. *bioRxiv* (2024). <https://doi.org/10.1101/2023.11.28.568945> arXiv:<https://www.biorxiv.org/content/early/2024/05/02/2023.11.28.568945.full.pdf>
- [40] Yajie Wang, Pu Xue, Mingfeng Cao, Tianhao Yu, Stephan T. Lane, and Huimin Zhao. 2021. Directed Evolution: Methodologies and Applications. *Chemical Reviews* 121, 20 (2021), 12384–12444. <https://doi.org/10.1021/acs.chemrev.1c00260> arXiv:<https://doi.org/10.1021/acs.chemrev.1c00260> PMID: 34297541
- [41] James T. Wilson, Riccardo Moriconi, Frank Hutter, and Marc Peter Deisenroth. 2017. The reparameterization trick for acquisition functions. arXiv:1712.00424 [stat.ML] <https://arxiv.org/abs/1712.00424>
- [42] Dengyong Zhou, Olivier Bousquet, Thomas Lal, Jason Weston, and Bernhard Schölkopf. 2003. Learning with Local and Global Consistency. In *Advances in Neural Information Processing Systems*, S. Thrun, L. Saul, and B. Schölkopf (Eds.), Vol. 16. MIT Press. https://proceedings.neurips.cc/paper_files/paper/2003/file/87682805257e619d49b8e0dfdc14affa-Paper.pdf

A Proofs

A.1 Proof of Proposition 1

PROOF. We compute the probability that z belongs to $\text{Conv}(\mathbb{X})$, $\mathbf{P}(z \in \text{Conv}(\mathbb{X}))$. By definition, we have $z = \sum_{i=1}^N \lambda_i x_i$ and $\beta \bar{x} + (1 - \beta)\epsilon = \sum_{i=1}^N \lambda_i x_i$, therefore,

$$\epsilon = -\frac{\beta}{1-\beta} \bar{x} + \sum_{i=1}^N \frac{\lambda_i}{1-\beta} x_i$$

WLOG, letting $\bar{x} = x_1$, we get that

$$\epsilon = \frac{\lambda_1 - \beta}{1 - \beta} x_1 + \sum_{i=2}^N \frac{\lambda_i}{1 - \beta} x_i.$$

This will lead to

$$\begin{aligned} \|\epsilon\|^2 &= \frac{\lambda_1 - \beta}{1 - \beta} \langle \epsilon, x_1 \rangle + \sum_{i=2}^N \frac{\lambda_i}{1 - \beta} \langle \epsilon, x_i \rangle \\ &\leq \underbrace{\frac{1 + \beta}{1 - \beta} \max_{i=1, N} |\langle \epsilon, x_i \rangle|}_{C_\beta}. \end{aligned}$$

Since all x_i are independent, for $x \in \mathcal{N}(0, I_d)$, we have that:

$$\mathbf{P}\left(\|\epsilon\|^2 \leq C_\beta \max_{i=1, N} |\langle \epsilon, x_i \rangle|\right) = 1 - \left(\mathbf{P}(\|\epsilon\|^2 \geq C_\beta |\langle \epsilon, x \rangle|)\right)^N.$$

For sufficiently large d , we approximate the Chi-square as the normal distributions

$$A, B, \|\epsilon\|^2 \sim \mathcal{N}(d, 2d).$$

Now let $S_1 = \|\epsilon\|^2 - \frac{C_\beta}{2} A + \frac{C_\beta}{2} B$ and $S_2 = \|\epsilon\|^2 + \frac{C_\beta}{2} A - \frac{C_\beta}{2} B$ are normal distribution with mean d and variances

$$\begin{aligned} \text{Var}(S_1) &= d(C_\beta^2 + 2) - \frac{C_\beta^2}{2} \text{Cov}(A, B) \\ &\quad - C_\beta \text{Cov}(\|\epsilon\|^2, A) + C_\beta \text{Cov}(\|\epsilon\|^2, B) \end{aligned}$$

Due to symmetric distribution of x , we can see that $\text{Cov}(\|\epsilon\|^2, A) = \text{Cov}(\|\epsilon\|^2, B)$, also, $\text{Cov}(A, B) = 0$ since ϵ and x are independent. The derivation is likewise for S_2 . Thus,

$$S_1, S_2 \sim \mathcal{N}\left(d, d(C_\beta^2 + 2)\right).$$

Since $\|\epsilon\|^2 - \frac{C_\beta}{2}|A - B| \geq 0$ iff $S_1, S_2 \geq 0$, we derive that

$$\begin{aligned} \mathbf{P}(\|\epsilon\|^2 \geq C_\beta |\langle \epsilon, x \rangle|) &= \mathbf{P}(S_1 > 0) \times \mathbf{P}(S_2 > 0) \\ &\approx \left(1 - \frac{\exp(-d/(2(C_\beta^2 + 2)))}{\sqrt{2\pi d}(C_\beta^2 + 2)^{-1}}\right)^2 \end{aligned}$$

Finally, we get that

$$\begin{aligned} \mathbf{P}\left(\|\epsilon\|^2 \leq C_\beta \max_{i=1,N} |\langle \epsilon, x_i \rangle|\right) &= 1 - \exp\left(2N \log\left(1 - \frac{\exp\left(-\frac{d}{2(C_\beta^2 + 2)}\right)}{\sqrt{2\pi d}(C_\beta^2 + 2)^{-1}}\right)\right) \\ &\approx 1 - \exp\left(-2N \frac{\exp\left(-\frac{d}{2(C_\beta^2 + 2)}\right)}{\sqrt{2\pi d}(C_\beta^2 + 2)^{-1}}\right) \end{aligned}$$

Thus for all $N \ll \exp\left(\frac{d}{2(C_\beta^2 + 2)}\right)$ and d sufficiently large, The left hand side is close to 0. As a conclusion,

$$\lim_{d \rightarrow \infty} \mathbf{P}(z \in \text{Conv}(\mathbb{X}_d)) = 0.$$

□

A.2 Proof of Proposition 2

PROOF. Since $D(z, \text{Conv}(\mathbb{X})) = \inf_{x \in \text{Conv}(\mathbb{X})} (\|z - x\|)$, for any $\bar{x} \in \text{Conv}(\mathbb{X})$, we should have:

$$\mathbb{E}[D(z, \text{Conv}(\mathbb{X}))] \leq \mathbb{E}\|z - \bar{x}\|.$$

Replace z by $\beta * \bar{x} + (1 - \beta) * \epsilon$, we obtain:

$$\begin{aligned} \mathbb{E}\|z - \bar{x}\| &\leq \mathbb{E}\|\beta * \bar{x} + (1 - \beta) * \epsilon - (1 - \beta) * \bar{x} - \beta * \bar{x}\| \\ &\leq \mathbb{E}\|(1 - \beta)(\epsilon - \bar{x})\| \\ &\leq (1 - \beta)(\mathbb{E}\|\epsilon\| + \mathbb{E}\|\bar{x}\|) \end{aligned}$$

We have that $\epsilon \sim \mathcal{N}(0, I_d)$ and $\bar{x} \sim \mathcal{N}(0, I_d)$. Chandrasekaran et al. [10] has shown that the upper bound of their expectations' norms is $\sqrt{d}$. Thus, we have

$$\begin{aligned} \mathbb{E}\|z - \bar{x}\| &\leq (1 - \beta)(\mathbb{E}\|\epsilon\| + \mathbb{E}\|\bar{x}\|) \\ &< (1 - \beta)(\sqrt{d} + \sqrt{d}) = 2(1 - \beta)\sqrt{d}. \end{aligned}$$

Therefore,

$$\mathbb{E}[D(z, \text{Conv}(\mathbb{X}))] < 2(1 - \beta)\sqrt{d},$$

which completes the proof. □

B Numerical Results For Other Protein Tasks

We present experimental results for additional difficulty levels of the GFP and AAV protein datasets. Statistics for each benchmark are in Table 6, and the experimental setup follows Section 4.1. Table 7 shows that our GROOT achieves state-of-the-art results across all difficulty levels, even with limited labeled data. Furthermore, the enhanced S-ReLSO, incorporating our smoothing strategy, significantly outperforms the original ReLSO, notably achieving an 8-fold

improvement in the GFP *medium* task. These results highlight the effectiveness of our framework.

Table 6: Statistic of benchmarks.

Task	Difficulty	Fitness Range (%)	Mutational Gap	Best Fitness	$ \mathcal{D} $
AAV	Easy	50 – 60th	0	0.53	5609
	Medium	20 – 40th	6	0.38	2828
	Hard	< 30th	7	0.33	2426
GFP	Easy	50 – 60th	0	0.79	4413
	Medium	20 – 40th	6	0.62	2139
	Hard	< 30th	7	0.10	3448

C Hyperparameters Tuning Process

For all tasks detailed in the main text, we keep the architecture of the VAE and the surrogate model, as well as the hyperparameters for the optimization algorithms, consistent. We then use the Optuna [1] package to tune the hyperparameters for each task. The range of hyperparameters is listed below:

- Number of nodes N : {4000, 5000, 6000, ..., 20000};
- Coefficient α : {0.1, 0.15, 0.2, ..., 0.9};
- Number of propagation layers m : {1, 2, 3, 4};
- Number of neighbors k : {2, 3, 4, ..., 8}

Table 8 shows our method's performance with optimal hyperparameters for each task. The AAV dataset performs best with a smaller graph (4,000 nodes) and fewer propagation layers ($m = 1$), while the GFP dataset requires larger graphs (14,000–18,000 nodes) and four propagation layers. Hyperparameters α and k remain consistent across difficulty levels within the same dataset. This suggests that while hyperparameter selection depends on dataset characteristics, it remains stable across difficulty levels, reducing the effort for tuning.

D Design-Bench

Tasks. To showcase the domain-agnostic nature of our method, we evaluate it on three tasks from Design-Bench [38], excluding noisy (e.g., ChEMBL) or expensive domains (e.g., NAS). D'Kitty and Ant are continuous tasks with input dimensions of 56 and 60, optimizing the structure of simulated robots for speed (Ant) and target location (D'Kitty). TF Bind 8 is a discrete task to find a length-8 DNA sequence maximizing binding affinity to SIX6_REF_R1. Its design space includes sequences of four nucleotide types. Design-Bench provides public and hidden datasets, along with exact oracles for evaluation. We simulate labeled data scarcity by subsampling 1% of each task's public dataset.

Baselines. We compare GROOT with four state-of-the-art models in the offline setting: MINs [23], BONET [28], BDI [11], and ExPT [30]. All baseline results are sourced from those reported by Nguyen et al. [30].

Evaluation. For each method, we allow an optimization budget of $Q = 256$. We report the median, max, and mean scores among the

Table 7: AAV and GFP optimization results for GROOT and baseline methods. The standard deviation of 5 runs with different random seeds is indicated in parentheses.

Method	AAV <i>easy</i> task			AAV <i>medium</i> task			AAV <i>hard</i> task		
	Fitness $\uparrow$	Diversity	Novelty	Fitness $\uparrow$	Diversity	Novelty	Fitness $\uparrow$	Diversity	Novelty
AdaLead	0.53 (0.0)	5.7 (0.4)	3.0 (0.7)	0.49 (0.0)	5.3 (0.7)	6.3 (0.4)	0.46 (0.0)	5.4 (1.7)	6.9 (0.9)
CbAS	0.03 (0.0)	23.2 (0.2)	17.5 (0.5)	0.02 (0.0)	23.1 (0.1)	18.3 (0.4)	0.02 (0.0)	23.0 (0.2)	18.5 (0.5)
BO	0.01 (0.0)	20.1 (0.5)	22.3 (0.4)	0.00 (0.0)	20.4 (0.2)	21.5 (0.5)	0.01 (0.0)	20.5 (0.2)	20.8 (0.4)
GFN-AL	0.05 (0.0)	18.2 (3.3)	21.0 (0.0)	0.01 (0.0)	16.9 (2.4)	21.4 (1.0)	0.01 (0.0)	14.4 (6.2)	21.6 (0.5)
PEX	0.44 (0.0)	6.33 (1.1)	4.0 (0.6)	0.35 (0.0)	6.86 (0.9)	4.2 (1.0)	0.29 (0.0)	6.43 (0.5)	3.8 (0.7)
GGs	0.49 (0.0)	9.0 (0.2)	8.0 (0.0)	0.51 (0.0)	4.9 (0.2)	5.4 (0.5)	0.60 (0.0)	4.5 (0.5)	7.0 (0.0)
ReLSO	0.18 (0.0)	1.4 (0.0)	5.0 (0.0)	0.18 (0.0)	15.7 (0.0)	11.0 (0.0)	0.03 (0.0)	18.6 (0.0)	15.0 (0.0)
S-ReLSO	0.47 (0.0)	8.6 (0.0)	4.0 (0.0)	0.26 (0.0)	13.7 (0.0)	7.0 (0.0)	0.2 (0.0)	8.3 (0.0)	11.0 (0.0)
GROOT	0.65 (0.0)	2.0 (0.7)	1.0 (0.0)	0.59 (0.0)	5.1 (2.3)	5.4 (0.6)	0.61 (0.0)	5.0 (1.0)	7.0 (0.0)

Method	GFP <i>easy</i> task			GFP <i>medium</i> task			GFP <i>hard</i> task		
	Fitness $\uparrow$	Diversity	Novelty	Fitness $\uparrow$	Diversity	Novelty	Fitness $\uparrow$	Diversity	Novelty
AdaLead	0.68 (0.0)	7.0 (0.4)	2.3 (0.4)	0.52 (0.0)	8.6 (3.0)	4.5 (0.5)	0.47 (0.0)	8.3 (1.9)	9.5 (2.7)
CbAS	-0.09 (0.0)	190.6 (1.5)	171.5 (4.6)	-0.08 (0.0)	171.5 (59.3)	200.8 (2.2)	-0.09 (0.0)	170.0 (29.2)	202.3 (0.8)
BO	-0.06 (0.0)	55.7 (1.3)	198.1 (5.6)	-0.04 (0.0)	58.1 (2.0)	199.8 (4.4)	-0.03 (0.1)	59.4 (2.0)	197.0 (7.6)
GFN-AL	0.27 (0.1)	83.7 (83.2)	222.7 (1.2)	0.27 (0.1)	65.5 (12.5)	223.0 (1.0)	0.19 (0.1)	54.4 (32.7)	224.0 (5.4)
PEX	0.62 (0.0)	6.6 (0.3)	3.9 (0.2)	0.46 (0.0)	7.8 (0.7)	4.6 (1.0)	0.13 (0.0)	12.6 (1.2)	7.1 (1.1)
GGs	0.84 (0.0)	5.6 (0.2)	3.5 (0.2)	0.76 (0.0)	3.7 (0.2)	5.0 (0.0)	0.74 (0.0)	3.0 (0.1)	8.0 (0.0)
ReLSO	0.40 (0.0)	3.6 (0.0)	6.0 (0.0)	0.08 (0.0)	29.0 (0.0)	18.0 (0.0)	0.60 (0.0)	38.8 (0.0)	8.0 (0.0)
S-ReLSO	0.72 (0.0)	108.3 (0.0)	2.0 (0.0)	0.64 (0.0)	32.8 (0.0)	6.0 (0.0)	0.80 (0.0)	10.5 (0.0)	7.0 (0.0)
GROOT	0.91 (0.0)	1.9 (0.3)	1.0 (0.2)	0.87 (0.0)	2.7 (0.1)	5.0 (0.0)	0.88 (0.0)	2.7 (0.4)	6.0 (0.0)

Table 8: Results of best selected hyperparameters for each task.

Task	Difficulty	N	m	α	k	Algorithm	Fitness	Diversity	Novelty
AAV	Harder1	4000	1	0.6	4	L-BFGS	0.56 (0.0)	6.2 (1.0)	13.0 (0.0)
	Harder2	4000	1	0.6	4		0.51 (0.0)	6.2 (1.6)	12.8 (0.5)
	Harder3	4000	1	0.6	4		0.45 (0.1)	9.4 (1.6)	13.2 (0.5)
GFP	Harder1	18000	4	0.2	4	Grad. Ascent	0.89 (0.0)	3.0 (0.2)	7.4 (0.6)
	Harder2	14000	4	0.2	5		0.87 (0.0)	3.1 (0.2)	7.6 (0.6)
	Harder3	14000	4	0.2	4		0.78 (0.1)	5.2 (2.4)	8.0 (0.0)

256 proposed inputs. Following prior works, scores are normalized to $[0, 1]$ using the minimum and maximum function values from a large hidden dataset: $y_{\text{norm}} = \frac{y - y_{\min}}{y_{\max} - y_{\min}}$. The mean and standard deviation of the scores are reported across three independent runs for each method.

attention heads, latent size 128) and a convolutional decoder (Section 3.2). For model-based optimization, we apply Gradient Ascent with a 0.005 learning rate for 500 iterations. Latent graph-based smoothing uses hyperparameters from Appendix C, tuned with Optuna for each task. Table 9 details the selected hyperparameters.

Table 9: Optimal settings of Design-Bench tasks.

Task	N	m	α	k
D’Kitty	16000	3	0.3	5
Ant	16000	3	0.3	5
TF Bind 8	14000	6	0.6	2

Implementation Details. For each task, we train the VAE on the full public dataset, using a 6-layer Transformer encoder (8